AirMatrix: synchronised low-latency audio over standard Wi-Fi and Ethernet

Public technical disclosure · version 2026-10-12 · Valiant Music, Brunswick Heads, Australia

> **Purpose.** This document publicly discloses the design of the AirMatrix audio transport (AMT), its clock synchronisation, node audio path, routing and mix engine, and related system ideas, so that they are available to the public as prior art from the date above. It describes work in progress; figures are labelled as measured, simulated or targets in the text.

Protocol spec v0.1

Draft wire protocol for the AirMatrix transport (AMT). It is built on RTP so standard tools (Wireshark, AES67 devices) understand it. Field names and sizes are proposals for Phase 0 to test.

Scope and key decisions

v0.1 covers what Phases 1–2 need: wired and Wi-Fi audio flows between Mac, Windows and Pi nodes, with clock sync, protection, telemetry and encryption. Matrix control, NMOS and comms build on it later.

#	Decision	Why
D1	RTP (RFC 3550) over UDP, IPv4 and IPv6, with our fields in an RTP header extension (RFC 8285)	AES67 compatibility; Wireshark and existing tools work; about 20 bytes of extra overhead, which is negligible
D2	Media timestamps come from the network clock (the AES67 method)	Every receiver knows exactly when each sample should play
D3	**One packet per packet time per path.** Protection goes inside the packet, not in extra packets.	On Wi-Fi, each packet costs a fixed ~100 µs of airtime whatever its size, so packet rate is the scarce resource, not bytes
D4	Layered redundancy inside each packet: full 24-bit for the current period, plus 16-bit and companded copies of earlier periods	A lost packet is recovered from the next one at reduced bit depth. This is how the 24→16-bit idea works at no extra packet rate.
D5	Multipath (two radios, or cable + Wi-Fi) sends identical packets on each path; receivers drop duplicates by sequence number	SMPTE 2022-7 semantics; no added latency
D6	Encryption with SRTP AEAD AES-128-GCM (RFC 7714) on AirMatrix flows; plain RTP on AES67 interop flows	A standard scheme, hardware-accelerated
D7	Clock: PTPv2 where available; otherwise AMT-Sync, our two-way time transfer with lucky-packet filtering	Laptops and Wi-Fi can't timestamp packets in hardware
D8	Retransmission (RTCP NACK, RFC 4585 / RTX, RFC 4588) only for the archive path, never for live playout	Perfect recordings without adding live latency
D9	48 kHz baseline, 96 kHz optional; 24-bit PCM	Broadcast norm

A correction from this spec work: earlier pages described the layers as *separate packets*. That would triple the packet rate on Wi-Fi, which is the one thing we can't afford. Per D3 and D4, layers travel inside one packet as redundancy for earlier periods. The cost is one extra packet time of latency (~2 ms) when the redundancy is used. Receivers on tight latency can skip it.

Stream and timing model

Every sample has a capture time on the shared network clock, and every receiver plays it at that time plus a fixed latency. That one rule gives sync across wired and wireless.

Terms

Term	Meaning
Node	Any device running AMT: Mac, Windows, Pi, Android, gateway
Flow	One sender's stream of 1–N channels (stereo for v0.1 wireless); one RTP SSRC
Subscription	A receiver taking a flow; the matrix is a set of subscriptions with gains
Packet time (P)	Audio duration per packet
Link latency (L)	Capture to analog output, set by the tier or sync group
Network time (T)	Shared clock, in TAI nanoseconds, from PTP or AMT-Sync

Packet times

Path	P	Frames per packet at 48 kHz
Wired	125 µs / 250 µs / 1 ms	6 / 12 / 48
Wi-Fi	**2 ms default**; 1 ms or 4 ms allowed	96 (48 / 192)

Timestamps (AES67 method). The RTP timestamp is the capture time of the packet's first sample, counted in samples on network time:

$$\mathrm{ts} = \left\lfloor T_{\mathrm{capture}} \cdot f_s \right\rfloor \bmod 2^{32}$$

T_capture is the moment the sample entered the converter: the sound card's buffer timestamp minus its reported input latency.

Playout. The receiver plays sample ts at network time ts / fs + L. It starts the sample early by its own output latency (buffer plus DAC), so the analog output lands exactly on time. The 32-bit timestamp wraps every ~24.9 hours at 48 kHz; receivers unwrap it against network time.

Deadline. A packet is useful only if it arrives before playout minus the receiver's processing margin (default 0.5 ms). The sender writes the remaining budget into each packet (see Packet format), so anything on the path can tell when a packet is stale.

Choosing L. Each subscription has an L from its tier, or from its sync group (the largest member requirement plus a margin). L changes only by click-free resampling ramps of under 0.1% (idea 7).

Packet format

An AMT packet is a standard RTP packet: one header extension carrying timing and path fields, then a payload of audio blocks. A default Wi-Fi stereo packet is 1,289 bytes, well under the 1,500-byte limit for a single unfragmented packet.

RTP header (RFC 3550, unchanged)

Field	Bits	Value
Version / padding / extension / CSRC count	2 / 1 / 1 / 4	2 / 0 / **1** / 0
Marker	1	1 on the first packet after a discontinuity
Payload type	7	Dynamic (96–127), declared in SDP
Sequence number	16	Per flow; **the same on every path** so duplicates can be dropped
Timestamp	32	Capture time in samples (Stream and timing model)
SSRC	32	Flow identifier

Header extension: `amt-core`

Uses the RFC 8285 one-byte form. The element ID is mapped in SDP to `urn:airmatrix:amt:core:v0`. Total extension: 16 bytes including the RFC 8285 header and padding.

Field	Bits	Meaning
version	4	0 for this draft
mode	4	Flags: bit-exact, redundancy present, compressed, test signal
path_id	4	Which path sent this copy (0–15)
slot	12	Soft time slot index assigned by the controller
budget_us	16	Microseconds left before the deadline, at send time
send_time	32	Sender network time at transmit, in µs (low 32 bits); gives one-way delay for margin meters

Payload: audio blocks

A 1-byte block count, then one 4-byte descriptor per block (type 8 bits, period offset 8 bits, length 16 bits), then the blocks. Period offset 0 is this packet's period, and 1 is the previous one.

Type	Name	Content
0x01	PCM24	24-bit PCM, big-endian, interleaved (as AES67 L24)
0x02	PCM16	Top 16 bits of each sample
0x03	BC10	Block-companded 10-bit: one 8-bit scale (a shift count) per channel per 1 ms block, then 10-bit packed samples
0x04	LLC	Lossless-compressed PCM24 (reserved for v0.2)
0x10	TEST	Tone or ident metadata (idea 30)

Layouts

Layout	Blocks	Use
Wi-Fi default	PCM24 @0, PCM16 @1, BC10 @2	Normal wireless flows
Wi-Fi tight	PCM24 @0, PCM16 @1	Smaller packets; covers one lost packet
Bit-exact	PCM24 @0 only, with multipath	Dolby E and other bitstreams
Wired	PCM24 @0 only	Wired flows; protection comes from 2022-7 dual networks
AES67 interop	Plain L24 payload, no extension, no encryption	Dante, Ravenna and Livewire+ devices

Size budget: Wi-Fi default, stereo, 48 kHz, P = 2 ms (96 frames)

Part	Bytes
PCM24 current period	576
PCM16 previous period	384
BC10 period before that	244
Block header	13
RTP + amt-core	28
SRTP auth tag	16
UDP + IPv4	28
Total	**1,289 (500 packets/s, ~5.2 Mbps per direction per path)**

Channel limit: the default layout fits **2 channels** per packet at P = 2 ms. More channels per wireless node need lossless compression (v0.2), a shorter packet time (more packets per second), or several flows. This is an open question for multichannel nodes.

Protection

Five mechanisms in order of cost: multipath (free in latency), in-packet redundancy (costs one packet time each step), concealment (always available), FEC (optional), and retransmission for the archive only.

Receiver decision for each period n, at playout time

1. **PCM24 for n** arrived on any path: play full 24-bit.
2. Otherwise, **PCM16 for n** arrived inside packet n+1: play 16-bit with TPDF dither.
3. Otherwise, **BC10 for n** arrived inside packet n+2: play the companded version.
4. Otherwise: **conceal** with Burg prediction and time-stretch (idea 27); fade only after about 20 ms of continuous loss.

Layer changes happen at period boundaries. No crossfade is needed: every copy of a period is the same waveform at a different bit depth, and 16-bit copies are dithered, so only the noise floor changes. Crossfades (32 samples) are used only around concealment, where the waveform itself is missing. See `07-node-audio-path.md`.

Protection depth depends on latency

Redundancy for period n arrives one or two packets later, so a receiver can only use it if its latency L leaves room. The receiver works this out from its own L.

L (Pi node on Wi-Fi, P = 2 ms)	Protection available
~6 ms (base)	Multipath only; one lost packet is concealed
~8 ms	+ one lost packet recovered at 16-bit
~10 ms	+ two lost in a row; the second at 10-bit companded

These base latencies are Phase 2 targets, not measurements.

Multipath (dual radio, two APs, cable + Wi-Fi)

- The sender sends identical packets (same SSRC and sequence number) on every path; only `path_id` differs.
- The receiver keeps the first copy of each sequence number, using a 256-packet dedupe window.
- Per-path arrival statistics feed the RF map and margin meters, so a path that is always second is still known to be healthy.

FEC (optional)

- FlexFEC (RFC 8627) parity packets, configured per flow.
- **Off by default on Wi-Fi,** because every parity packet costs the same airtime as an audio packet. In-packet redundancy is cheaper there.
- Useful on lossy wired links and internet gateways.

Archive retransmission (idea 6)

- Every sender keeps a ring buffer of at least 60 s of its packets.
- The recorder requests missing sequence numbers with RTCP generic NACK (RFC 4585), and requests a full-quality resend for periods that were played from PCM16 or BC10.
- The sender answers with RTX packets (RFC 4588) on a separate SSRC at **best-effort priority**, so retransmission never competes with live audio.
- The recorder writes Broadcast WAV only after a window closes complete, or marks any gaps that could not be filled.

Clock sync

One wired node leads network time. Every other node measures its offset from the leader with AMT-Sync, steers a model of network time, and resamples its sound card into that time. The v0.1 target is ±20 µs on wired and ±50–100 µs on Wi-Fi.

Hierarchy

See `05-clock-sources.md` for accepted external sources, the clock-class ladder, leader election and the free-running stability work. In short: every leader announces a class (external locked, holdover, internal, backup) and a priority in each response; followers prefer the best; a backup that was following the leader takes over without a step.

1. If a facility PTP grandmaster exists (ST 2059-2 / AES67 profile), the leader locks to it with linuxptp, using hardware timestamps where the NIC supports them.
2. Otherwise the leader is a wired node chosen by the controller (normally the engine host), with a designated backup leader.
3. Every other node follows the leader through AMT-Sync, or through PTP directly on wired nodes with hardware timestamping.

AMT-Sync messages (UDP, marked for the Wi-Fi voice queue)

Message	Fields
SYNC_REQ	node_id, seq, t1 (node local time at send)
SYNC_RESP	seq, t1 echoed, t2 (leader receive time), t3 (leader send time), leader_id, leader_quality

On the wire (UDP, default port 5005, DSCP 46) SYNC_REQ is 28 bytes and SYNC_RESP 48 bytes; the layouts are in `amt-core::syncmsg`. The node records t4 when the response arrives. Timestamps come from the kernel where possible (Linux `SO_TIMESTAMPING`; equivalent APIs on Windows and macOS), otherwise from the application.

$$\text{offset} = \frac{(t_2 - t_1) + (t_3 - t_4)}{2} \qquad \text{delay} = (t_4 - t_1) - (t_3 - t_2)$$

Lucky-packet filtering

Wi-Fi queues only ever *add* delay, so the fastest exchanges are the most accurate.

- **Burst:** every 250 ms, send 4 requests spaced 1–3 ms apart.

- **Keep:** the exchange with the lowest delay in each burst.
- **Window:** over a 2–4 s sliding window, discard any sample whose delay exceeds the window minimum plus a threshold (default 200 µs).
- **Servo:** an alpha-beta tracker (equivalent to a PI loop) estimates offset and frequency, with high gains for the first 16 updates and low gains after. The published clock slews toward the estimate (within ±200 ppm of the estimated frequency) and steps only at first lock or if the error exceeds 1 ms.
- **Lock:** reported when the estimated clock error (half the RMS of the last 16 servo errors, because the servo averages away most of the measurement noise) is below 50 µs, and lost only above 100 µs, so a noisy but healthy link does not flap.
- **Asymmetry:** if Wi-Fi uplink and downlink delays differ, the offset is biased by half the difference. Phase 0 measures this per node type (using the audio loopback probe) and stores a calibration value.

Media clock and ASRC

- **Free-running sound cards** (most USB interfaces and laptops): the node measures the card's sample rate against network time, giving a ratio in ppm. A high-quality resampler converts between the card and network time, and its servo holds the buffer fill at target. Ratio changes are slewed so they are inaudible.
- **Lockable clocks** (word clock input, or a Pi HAT codec with an adjustable PLL): steer the hardware clock directly instead. No resampling, so best quality.
- **Resampler quality target:** transparent at 24-bit, at least 120 dB THD+N.
- **Holdover:** if sync is lost, the node keeps running on its last frequency estimate and flags the condition. Drift over 60 s at 1 ppm is 60 µs, still inside every tier's margin.

Transmission

Each node sends exactly one packet per path per packet time, at its own assigned moment, and never sends a packet that can no longer arrive in time.

Soft time slots (idea 4)

- **Cycle:** network time is divided into cycles of length P, starting at multiples of P.
- **Slot offsets:** the controller assigns each node on an AP an offset o, spread evenly: o = i × P / N for N nodes, with a small guard time.
- **No added latency:** each node shifts its *audio period boundaries* by its own offset, so a packet's audio is complete exactly when its slot arrives. Timestamps carry the true capture time, so playout is unaffected.
- **Downlink:** the engine paces packets to each node at the node's downlink slot, half a cycle away from its uplink slot.
- **Beacons:** APs send a beacon about every 102.4 ms, which occupies the air briefly. v0.1 learns beacon timing from periodic delay spikes and nudges slots away from it. Linux nodes can read it from the driver later.

Deadline drop (idea 5)

- Before sending, the node checks budget_us against an air-time estimate. Packets that can't make it are dropped and counted.
- Socket send buffers are kept to 2–4 packets, so stale audio can't queue up.
- **Linux:** launch-time scheduling (`SO_TXTIME` with the ETF queue discipline), airtime queue limits in the Wi-Fi driver, and an optional lower retry limit on dedicated nodes.
- **macOS / Windows:** application-level pacing and small buffers.

QoS marking

Traffic	DSCP	Wi-Fi queue (WMM)
AMT media on Wi-Fi	46 (EF) by default	Voice (AC_VO), per RFC 8325 mapping
AMT-Sync	46 (EF)	Voice
AMT and AES67 media on wired	34 (AF41), per AES67	—
PTP	46 (EF), per AES67	—
Archive retransmission	0 (best effort)	Best effort
Control	16 (CS2)	Best effort / background

A router gotcha: many consumer routers use a simple mapping (the top 3 bits of DSCP), which puts EF into the *video* queue, not voice. The network wizard measures which queue we actually land in and, if needed, switches wireless media to DSCP 48 (CS6), which lands in voice on those routers.

Platform APIs: macOS sets `SO_NET_SERVICE_TYPE` to voice. Windows needs a QoS policy or the qWAVE API for DSCP to be honoured, which the installer configures. Android sets the socket traffic class and holds the low-latency Wi-Fi lock.

Control, security, telemetry and interop

The control plane is separate from audio and runs over TLS. Phase 1 can run without a controller ("peer mode"), so two computers can test the transport before any matrix exists.

Discovery

- Nodes advertise `_amt._udp` over mDNS/DNS-SD. TXT records carry node ID, role, capabilities (channels, rates, paths, platform) and key fingerprint.
- The controller advertises `_amt-ctl._tcp`.
- **Peer mode (Phase 1):** two nodes are paired by a config file or command line. One acts as sync leader.

Control channel

- Node ↔ controller over WebSocket on TLS 1.3, with Protocol Buffers messages.
- **Core messages:** `hello`, `adopt`, `flow_create`, `subscribe` (flow, L, layout, paths, slot), `set_gain`, `set_slot`, `telemetry`, `identify`.

- Losing the control channel never stops audio. Nodes keep their last subscriptions (Reliability principles).

Security

- **Identity:** each node generates an Ed25519 key pair on first start. Its fingerprint is shown as a QR code or LED pattern for adoption (idea 9).
- **Flow keys:** the controller issues per-flow SRTP master keys over the TLS channel and rotates them on a schedule. A key index in each packet allows hitless rotation.
- **Admission:** packets from unadopted nodes or with failed authentication are dropped and raise an alert.
- **Peer mode:** a shared key from the config file.

Session description

Each flow is described in SDP (RFC 8866), in AES67 style: `ptime`, `ts-refclk`, `mediaclk`, plus `a=extmap` for amt-core and an AMT layout attribute. The same description serves SAP announcements for interop and NMOS IS-05 later.

Telemetry (every 1 s, per flow and per path)

Side	Metrics
Receiver	Received, duplicates, late, lost; periods played at 24-bit / 16-bit / 10-bit / concealed; arrival margin at the median, worst 1% and worst 0.1%
Sender	Deadline drops, send-queue depth, slot timing error
Sync	Offset, delay, window minimum, servo frequency, holdover state
Wi-Fi (where the OS exposes it)	Signal strength, data rate, retries, channel use

Telemetry goes to the controller over the control channel. RTCP receiver reports are also sent for interop. The black box (idea 13) keeps the raw per-packet records.

AES67 interop

- Any node or gateway can publish a flow as AES67 (L24, 48 kHz, 1 ms, multicast, SAP/SDP) and subscribe to AES67 flows.
- This includes Dante devices in AES67 mode.
- AES67 needs PTP, so the sync leader also acts as PTP grandmaster, or follows the facility's.

Open questions and Phase 0 experiments

Every uncertain number in this spec has a cheap experiment attached. All of them run on computers and the router you already have.

#	Question	Experiment	Pass criterion
E1	What does delay really look like on your Wi-Fi?	UDP probe at 500 packets/s with 1,289-byte packets for 1 hour: laptop to wired PC, both directions, idle and under load	A delay histogram with the median, worst 1% and worst 0.1%; periodic spikes identified (beacons, scans, AWDL)
E2	Is in-packet redundancy worth its bit-depth cost versus plain duplication?	Replay E1's recorded loss and delay traces through a simulator of both schemes at equal packet rate	Same concealment as plain duplication at equal depth, by construction (tested); the experiment instead measures the byte saving against any loss/jitter penalty from packet size (E1 re-run per packet size)
E3	How accurate is AMT-Sync over Wi-Fi?	**Click alignment test:** a wired and a wireless node each play a click at the same network time; both outputs are recorded on one stereo interface	Measured sync error at most ±100 µs; asymmetry bias measured
E4	Do soft slots reduce delay spikes?	Several senders (laptops, or processes on separate machines) with slots on and off	Worst-0.1% delay improves measurably
E5	Which WMM queue do we land in?	Send DSCP 46 vs 48 alongside bulk traffic and compare delay	The wizard rule for choosing DSCP is confirmed
E6	Is BC10 good enough as a last resort?	Measured SNR on music and speech, plus a blind listening comparison against ADPCM	Clearly better than concealment; no audible artefacts in 2–10 ms bursts
E7	Which resampler?	Compare rubato (Rust, MIT), soxr and speexdsp on THD+N and CPU (Mac, Windows, Pi 5)	At least 120 dB THD+N within the CPU budget
E8	Smallest reliable audio buffers per platform?	cpal (CoreAudio, WASAPI, ASIO, ALSA) at 32/64/128 frames over a 1-hour run	Zero xruns at the chosen size on each platform
E9	How many channels per Wi-Fi node?	Lossless-compression ratio on real stage recordings	Decide the multichannel layout for v0.2

Decisions needed from you

- [ ] Protocol name: "AMT" (AirMatrix Transport) is the working name, but it clashes with two existing names (see below)

Name clash: "AMT" is already Intel's Active Management Technology and an IETF protocol, Automatic Multicast Tunneling (RFC 7450). A network protocol needs a name of its own; pick before going public.

Clock sources and free-running stability

Requirement : AirMatrix must accept external clock sources, and must be super stable without one.

All results below marked *(simulated)* come from `amt-sync simulate` / `run_failover` with a modelled network; nothing here has been measured on real Wi-Fi yet (see E3).

Two different clocks

Keeping these apart is the whole design.

	Network time	Media clock
What	The time all nodes agree on; stamps every packet	The crystal driving one converter
Who sets it	The elected leader	That node's own oscillator, or an external word-clock input
Reaches the converter?	**Never directly**	Yes

A node's converter always runs from its own clean clock (or word clock). Network time only sets the *resampler ratio* between the converter and the network, and that ratio is filtered slowly, so noise on the network clock cannot reach the converter as jitter. Wireless wobble stays in the packet layer.

Accepted sources

A source is anything that can act as the leader's time: it implements `TimeSource` and announces a clock class. `ClockModel` and `SharedClock` carry whichever time results to the audio thread.

Source	Gives	Typical quality	How it plugs in	Status
PTP grandmaster (AES67 / ST 2059-2)	Phase, frequency, absolute time	sub-µs wired	linuxptp (`ptp4l` + `phc2sys`) disciplines the OS clock; leader reads it	Planned (HANDOFF T2b)
GPS with PPS	Phase, frequency, absolute time	sub-µs	PPS into chrony/linuxptp, as above	Planned
Word clock / AES3 / MADI input on an audio interface	Frequency (and sample phase)	Interface-grade	The interface's sample counter, mapped to local time through stream timestamps, becomes the leader's `TimeSource`	Planned (needs T3 audio I/O)
LTC / MIDI timecode	Absolute *time of day* only	Frame-level, jittery	Aligns the timeline to timecode; **not** a sample-rate reference	Planned
NTP / system time	Absolute time	milliseconds	Not used for sync; may label the timeline	n/a
Internal oscillator	Frequency only	See below	`MonotonicClock`	**Built**

Choosing the leader

Every leader announces `(class, priority, id)` in each sync response (`amt-core::syncmsg`). Followers prefer the lowest:

Class	Value	Meaning
`EXTERNAL_LOCKED`	0	Locked to an external reference
`EXTERNAL_HOLDOVER`	64	Reference lost; free-running on its last good frequency
`INTERNAL`	128	Free-running on its own oscillator
`BACKUP`	192	Following another leader, ready to take over
`UNUSABLE`	255	Must not be followed

`amt-sync::election::Selector` implements the rules, all unit-tested:

- Lowest class wins, then lowest operator priority, then lowest id.
- **No preemption on ties:** a follower already on a leader switches only to one strictly better by class or priority. A different id on equal footing never causes a switch (no flapping).
- **Timeout failover:** a silent leader is dropped and the best remaining one used.
- A leader whose reference is lost (class `0 → 64`) is kept over an `INTERNAL` rival; one that degrades to `UNUSABLE` is replaced.

The leader's class can change while it runs (`LeaderInfo::set_class`), which is how a GPS or PTP loss is announced.

Stable without an external clock

Without a reference the network's *absolute* time is meaningless, so stability means two things: (1) nodes stay aligned with each other, and (2) a loss of the leader or of the network does not disturb them. What was built for that:

1. **Frequency from a long regression.** Single bursts carry tens of µs of queueing noise, and a servo that learns frequency from them wanders 1–2 ppm. Once 30 s of measurements exist the frequency comes from a least-squares line through the last 64 s, which is about ten times steadier. Frequency is what decides holdover: **1 ppm of error is 60 µs per minute.**
2. **A step keeps the frequency.** If the leader's time jumps, the offset is re-learned but the crystals' relative rate is not forgotten.
3. **Level-shift detection.** After a roam the path asymmetry can change, moving the measured offset by 100 µs or more for good. The regression window is cleared so this is absorbed as an offset, not misread as a frequency change.
4. **A published clock that finishes its corrections.** It slews toward the estimate (never backwards, at most ±200 ppm around the estimated frequency), then holds the servo's own frequency. The first version kept the corrective slew running through a blackout and drifted 14 ms; found by test and fixed.

5. **Hitless leader takeover.** A backup that has been following the leader answers with its own published clock (`NetworkTime`), which continues the same timeline. Followers switch to it without a step.

Measured in simulation *(simulated)*

Busy 5 GHz Wi-Fi-like path (400 µs each way, 150 µs mean queueing, 1% loss), truth known:

Scenario	Result
Steady, after 10 s	p99 error 13–25 µs
Frequency estimate, steady crystal	within 0.2 ppm
60 s total blackout (nothing arrives)	error ≤ 17 µs throughout (the previous servo: 205 µs)
Crystal warming at 1 ppm/min (a hard transient)	p99 15–23 µs
Typical room, 0.3 ppm/min, plus a 60 s blackout	error under 5 µs, re-locks cleanly
Leader dies, backup takes over (follower B)	no step, never backwards, error 7–27 µs before, during and for 70 s after

Slew horizon 8 s (was 1 s), 2026-10-10. Each 250 ms update aims the published clock at the servo's estimate over the horizon; a short horizon turns measurement noise straight into rate steps. Same simulated runs, worst 10 s window after 20 s: steady 11.4 µs (was 21.8), 60 s blackout 16.5 (22.3), 1 ppm/min warming 17.3 (23.3), 0.3 ppm/min + blackout 15.4 (23.0), failover 16 (28), still no step and never backwards. Clock wander within 100 ms on Wi-Fi: 95 ns plain / 20 ns after removing a straight line (was 551 / 166): about 18 dB smoother. `amt-sync simulate --horizon-s`, and the ignored test `smoothness_sweep` in `sim.rs`.

Holdover budget (what frequency error costs once replies stop):

Frequency error	Drift per minute
0.1 ppm	6 µs
0.5 ppm	30 µs
1 ppm	60 µs
5 ppm	300 µs

For scale, one sample at 48 kHz is 20.8 µs.

Measured on real Wi-Fi (2026-10-10)

The first field traces (phone to laptop through a router and a Wi-Fi extender: round-trip floor 3.7-8 ms, p99 27-88 ms) replayed through the follower (`amt-sync replay`, `crates/amt-sync/tests/field_traces.rs`):

- The old servo stepped the clock whenever one accepted burst was 400 us off: **13 and 53 steps** in 1 and 5 minutes. Steps are what audio hears.
- Widening the lucky-packet gate made it worse (more noisy bursts in, more steps, larger errors): the gate stays at 200 us.
- Fix: a step needs **3 consecutive errors beyond the threshold, all the same sign** (a real jump shows in every measurement after it; noise does not; unconfirmed ones are held, changing nothing), and **both thresholds scale with the path's measured noise** (step at least 6x the RMS servo error, level shift at least 3 standard errors of its 6-error mean), so a clean path keeps 400 us / 80 us and corrects a real 600 us jump within 2 s.
- Result on the same traces: **2 steps each (start-up only); worst distance from the hindsight reference 0.41 ms and 0.83 ms** (was 0.70 and 4.67 ms). A third trace, the phone on the router alone (no extender): **50 steps -> 2; frequency +6.6 -> +28.3 ppm (hindsight +28.6); worst 0.78 -> 0.26 ms**. Simulated clean paths unchanged (a 600 us leader jump: corrected in under 2 s, 5 us after).
- Not yet: the follower never reports Locked on such a path (the 50 us lock threshold is below its noise); the lock criterion should scale too (A2).

Rules for real hardware

- **Use an unslewed clock.** macOS (`mach_absolute_time`) and Windows (QPC) are unslewed. On **Linux the default monotonic clock is slewed by NTP/chrony** by up to 500 ppm, which would shift a node's apparent frequency under the servo. Use `CLOCK_MONOTONIC_RAW` (HANDOFF T2b; a safe API such as `rustix` avoids `unsafe`).
- **Pick the leader deliberately:** the best oscillator, least loaded machine. Normally the wired engine host with a pro audio interface. Better still, derive the leader's time from that interface's sample clock (planned): pro interfaces usually have a better crystal than the motherboard.
- **Run the sync thread at elevated priority.** Under heavy CPU load, userspace timestamps get noisy: 10 busy processes pushed the first-8-seconds frequency estimate off by 5–11 ppm on the test Mac (Intel, macOS). Kernel timestamps and a high-priority thread are the fixes (HANDOFF T2b).
- **Keep devices powered and warm.** Oscillators settle with temperature; the regression follows slow drift (tested to 1 ppm/min) but not step changes.

Mapping to code

Concept	Code
Class and priority on the wire	`amt-core::syncmsg` (`class` module, `SyncResp`)
Election	`amt-sync::election::Selector`
Leader announcement, runtime class change	`amt-sync::net::LeaderInfo`
Backup leader's time	`amt-sync::clock::NetworkTime`
Servo, regression, step/shift handling	`amt-sync::servo`
Clock that finishes corrections and holds frequency	`ClockModel` (two segments), `Follower::publish`
Failover simulation	`amt-sync::sim::run_failover`; `amt-sync simulate --failover-at 60`

Acceptance for "stable without a reference"

1. **24 h free-run soak** on the real two-machine pair: record the E3 click offset at the start and end, with the leader unchanged. Target: the offset changes by under ±100 µs. (An uncorrected 1 ppm error would grow by 86 ms in a day, so a pass shows the servo is continuously correcting, not just holding.)
2. **Chaos:** kill and restart the leader repeatedly; unplug the network for 60 s; roam the Wi-Fi node. No audible event on any node, and no step on followers of a backup leader.
3. **Reference loss:** with an external source built, drop the reference and confirm the class announcement changes to holdover and followers stay put.

The node audio path

How audio gets from one sound card to another, on time, through AMT. Built in `crates/amt-node`; everything here is measured in simulation with known truth (`tests/node_end_to_end.rs`, `tests/asrc_end_to_end.rs`), not yet on hardware.

The chain

```
SENDER   ADC (own crystal) → ASRC: read card audio at network-time positions
         → periods (2 ms) → lock-free queue → packetizer (24 + 16 + 10-bit
           layers) → socket
NETWORK  delay, jitter, loss
RECEIVER socket → depacketizer (dedupe, unwrap, decode) → lock-free queue
         → jitter buffer (best copy per period, dither, concealment)
         → ASRC: read network audio at the DAC's positions → DAC (own crystal)
```

Both ASRCs are the same component: a band-limited interpolator reading a ring buffer at positions chosen by a phase-locked tracker. The sender's tracker aims "network sample `s` reads card position `c(s)`"; the receiver's aims "DAC frame at network time `t` reads network sample `(t − L)·fs`". So the sample captured at network time `T` leaves the DAC at exactly `T + L`, whatever either crystal does.

Audio-thread code (jitter buffer render, ASRC render, sender period cutting) does no allocation and takes no locks; the tests run it inside an allocation trap.

Measured

	Result
Interpolator SNR, 64 taps (default)	~110 dB to 19 kHz; 128 taps 126-136 dB; 32 taps ~90 dB
Kernel latency per stage	32 / 64 / 128 taps: 0.33 / 0.67 / 1.33 ms
One ASRC, card 37 ppm off, +-1 sample timestamp jitter	109 dB, sync p99 2.1 us
Whole chain, perfect timestamps	**118 dB, lands within 0.1 us of capture + L**
Whole chain, +-1 sample timestamp jitter	116 dB with slow wander removed (78 dB raw), +-2.3 us
Whole chain, Wi-Fi-like (1 ms + 0.4 ms jitter, 2% loss), L 12 ms	102 dB, 4 concealments in 40 s, 0 late
End-to-end latency with 64-frame card buffers, clean network	**8 ms** with nothing late
Whole chain at the **timestamp jitter measured on the test Mac (Intel, macOS)** (+-2.2 samples, E8)	110 dB slow-wander removed (69 dB raw), timing -8.3..+3.2 us

Why raw SNR and "slow wander removed" differ. Sound-card timestamps carry jitter; the trackers average it, but a few microseconds of very slow phase wander (below about 10 Hz) remain. That is a pitch wander of about 0.003 cents: inaudible, and excluded by the notch of a THD+N measurement. Perfect timestamps remove it entirely (118 dB raw). So **timestamp quality per platform matters**: T3 must measure it, and backends with noisy timestamps (ALSA period interrupts) need a timestamp smoother in front of the tracker.

Latency budget

The DAC callback renders a block ahead of play, the ASRC needs half its kernel of look-ahead, and the sender can only cut a period once its last sample is in and past the kernel. So:

```
L ≥ sender (card block + period + kernel)
  + network delay at the percentile you design for
  + receiver (card block + callback lead ≈ one block + kernel)
  + redundancy depth × period (only if repairs must be usable)
```

Card buffer	**Fixed part (2 ms periods, 64-tap kernels)**	**Clean wired**	**Wi-Fi (p99.9 ~3 ms)**
64 frames (1.3 ms)	about 7.3 ms	**8 ms (measured)**	~11-13 ms
128 frames (2.7 ms)	about 11 ms	~12 ms	~15-17 ms
480 frames (10 ms)	about 34 ms	~35 ms (40 ms measured clean)	~40 ms

Card buffer size dominates. The same network gives 8 ms or 40 ms depending on the sound card setting. E8/T3 must establish the smallest reliable buffer per platform; on Windows that means ASIO or WASAPI exclusive mode.

Behaviour decided while building

- **The tracker low-passes its phase error** (two poles at 10× the loop bandwidth) before acting. The first version passed timestamp jitter straight into the playback rate (370 ppm block to block: flutter). Loop: 1 Hz to acquire for 3 s, then 0.1 Hz. Slower loops trade absolute timing for purity (0.05 Hz: 130 dB but timing wander up to 100 us under harsh jitter); 0.1 Hz is the balance.
- **Layer changes need no crossfade**: every copy of a period is the same waveform at a different bit depth; 16-bit copies are TPDF-dithered. Crossfades are used only around concealment. (The spec's 32-sample crossfade rule was written before this; the spec is updated.)
- **Concealment v0.1**: point-reflection continuation of the waveform under a 32-sample raised-cosine fade, silence, then a 32-sample fade-in. Never a click. Burg prediction (idea 27) is the upgrade.
- **Start-up** fades in from silence; periods before the first audio are counted as *waiting*, not loss.
- **Late copies** are split: late full-quality copies mean the budget is too small; late redundant copies are normal (they repeat periods already played).
- **Missing input is never invented**: the ASRC reports starved frames as silence.

Measured on real hardware (E8)

`amt-io timing` on this Intel Mac's built-in output: timestamp jitter p99 about 2 samples; callbacks on time even at 32 frames; the hardware runs at 44.1 kHz behind a 48 kHz stream (CoreAudio converting); and **cpal's playback timestamp omits output latency**, so a per-device latency calibration is required before nodes on different devices can align to the microsecond. Details: `docs/experiments/results/E8-2026-10-09-mac-builtin.md`.

Next

1. **Per-device output/input latency calibration** (loopback cable, or OS-reported latency): without it, "exactly T + L" is exact only up to each device's own latency.
2. **T3, sound-card glue** (cpal): run sender and receiver on real hardware, measure timestamp jitter, buffer sizes and true latency with a loopback cable.
3. A **timestamp smoother** for backends whose timestamps are noisy.
4. **Multipath send** (two sockets/interfaces, same packets, path ids).
5. **SRTP** (AES-GCM) on the packets.
6. **Burg concealment**.

Matrix engine core (`crates/amt-matrix`)

Any source to any destination with a gain on every crosspoint; buses that sum and feed other buses; mix-minus. One engine behind the matrix, mixer and comms views (spec, "Matrix and mix engine").

How it works

- **Control half** (UI/network thread) validates every change and compiles a small plan: buses in dependency order, then outputs, each a list of terms.
- **Audio half** mixes; allocation-free and lock-free (tested under an allocation trap while plans swap). Old plans return through a queue that can never be full, so the audio thread never frees memory.
- **Gains live in fixed slots** that outlast plan swaps. Every gain move, new route, removed route and mix-minus change is a linear ramp (10 ms default). A removed route stays in the plan, fading, until the audio thread has certainly played its last zero sample; only then is it dropped.
- **Batches** (`apply`): a scene or salvo is validated as its end state and travels as one message, switching on one sample. A single fader move is a lighter message of its own.
- **Loop detection** refuses a route that closes a loop, naming the loop.
- **Mix-minus** is a property of a feed from a bus: `bus − own`, where `own` is the participant's contribution evaluated with the bit-identical ramped gain the bus used. O(1) per listener instead of O(N). A change that would let the participant reach the bus by another path is refused, naming that path (no silent own-voice leak).

Measured (Intel i7-5775R, 2015)

Test	Result
Mix-minus error vs the ideal "everyone else" mix, 8 callers	-141 dBFS steady, -142 dBFS while the caller's own fader moves
Random routing churn, 3 000 blocks, DC inputs	largest sample step < 0.06 (ramps off: >= 0.25, so the check can see a click)
64 x 64, 16 buses, 499 crosspoints	8.4 us per 64-frame block (0.6% of a core)
256 x 256, 64 buses, 3 970 crosspoints	78 us (5.9%)
512 x 512, 64 buses, 11 643 crosspoints	246 us (18.5%)

Cost scales with active crosspoints, not N x M.

Not yet

Delay compensation per path (needed once plugins exist), metering taps, SIMD inner loops, persistence of scenes, permissions and locks, the network glue that turns node flows into matrix inputs and outputs.

Web streams as matrix sources

Requirement : any web stream can be added as an audio source to the matrix.

Built so far: `crates/amt-ingest` (library and CLI). Tested live against an AAC, an MP3 and a lossless FLAC-in-Ogg radio stream over the internet, and offline against a local HTTP server (see `docs/experiments/E4-web-stream-ingest.md`).

What a web stream is, to the matrix

A **non-real-time, unprotected source**:

- It runs on its encoder's clock, not ours (rate differs by some ppm).
- It arrives in bursts, and servers send many seconds at connect.
- It carries seconds of inherent delay. We add a buffer on top (default 1.5 s) and call the result the source's *delay*.
- It has no AMT layering, multipath or timestamps. Its audio enters the matrix as an ordinary flow, stamped on network time at the point it is decoded.

It is therefore the highest-latency tier, shown as such by the margin meters (idea 11), and unsuitable for lip-sync-critical paths. Because its delay is known, a stream can be placed in a sync group by delaying the other members by the same amount. Sample-accurate alignment additionally needs the node's asynchronous resampler to slave the stream's rate to network time (backlog).

Pipeline

```
URL ──► source (HTTP, redirects, .pls/.m3u, reconnect + backoff)
    ──► icy (strip in-band titles, report "now playing")
    ──► adts (AAC only: clear the MPEG-2 ID bit symphonia rejects)
    ──► decode (probe, decode, map to stereo)
    ──► resample to 48 kHz
    ──► elastic buffer (absorbs bursts; trims the connect burst)
    ──► matrix source (48 kHz stereo)
```

Formats

	Status
MP3, AAC-LC (ADTS, including the MPEG-2 flavour many radio servers send), Ogg Vorbis, FLAC (native and in Ogg), WAV/PCM, ALAC, ADPCM	Supported
`.pls` and simple `.m3u` playlists, redirects, HTTPS, in-band titles	Supported
Mono (duplicated to both sides); more than two channels (front pair used; proper downmix is backlog)	Supported
Opus, HE-AAC (`aacp`)	**Not yet.** Common on radio. Opus needs libopus (BSD); HE-AAC needs an SBR-capable decoder. A clear error is shown.
HLS / DASH (segmented), RTMP, SRT, RTSP, WebRTC, NDI	**Not yet.** HLS is detected and refused with a clear message.

Behaviour a live source needs (built)

- **Reconnects** with exponential backoff (1, 2, 4 ... 30 s). A session that played for 5 s or more resets the backoff.
- **Fatal errors stop it with a reason** instead of retrying forever: HTTP 4xx, an HLS manifest, a web page instead of audio, a codec we lack (on the first connection).
- **Trims the connect burst** so the delay returns to the target.
- **Reports honestly**: dropouts, reconnects, damaged packets, buffer fill, signal levels. The stream clock offset is shown only when a run is long enough to measure it (about 150 s).

Decisions and rules for the product

- **Runs as its own node, not inside the engine.** It parses untrusted data from the internet, so in the Broadcast profile it is a separate process that publishes an AMT flow, like any gateway (MADI, AES3). A decoder crash then cannot take the engine down. (Rust is memory-safe, but parsers still panic.)
- **URLs are untrusted.** Only `http`/`https`; playlists capped at 64 KiB and three hops. When a controller fetches URLs entered by users on a venue network, private and loopback addresses must be blocked unless an operator allows them (server-side request forgery). Not enforced yet.
- **Fallback.** A stream going away must not leave a hole on air: each stream source gets an optional backup (a second URL, or a looped file) and a silence alarm. Backlog.
- **Rights.** Using a stream, and above all recording it, is the operator's responsibility. Web sources are excluded from the live-and-archive recorder (idea 6) by default.
- **Loudness.** Streams vary by many dB. The ingest should measure loudness (EBU R128) and offer optional normalisation per source. Backlog.

Backlog (in HANDOFF as T6)

Opus and HE-AAC; HLS; SRT; stream-as-a-node publishing an AMT flow; asynchronous resampling to network time; backup URLs and silence alarm; loudness; SSRF allow-list; the reverse direction (matrix output encoded to an Icecast server).

Design ideas

Twenty-eight ideas that make AirMatrix more than a wireless Dante: each with how it works, why it matters, and build notes. Ideas 1–22 are summarised on the main tab; 23–28 come from a review of other protocols, at the end of this tab.

Wireless transport

These ideas make standard Wi-Fi behave like a scheduled, protected link. Together they set the latency floor and the glitch rate.

1. Graceful bit-depth degradation

- **How:** each packet carries its own period at 24-bit, plus a 16-bit copy of the previous period and a 10-bit companded copy of the one before. One packet per period keeps the packet rate low (see the Protocol spec tab).
- **L0 must add no delay.** A normal codec such as Opus adds about 5 ms, too much for a 5 ms tier. Use NICAM-style block companding instead: about 10 bits with a gain value per 1 ms block, the method behind broadcast TV stereo. It adds no delay beyond the block and sounds near-broadcast quality.
- **Switching:** layer changes crossfade over about 1 ms with dither, so the noise floor never steps.
- **Telemetry:** every route logs how often each layer was used, which feeds the margin meters (idea 11).

2. Two-AP seamless paths

- **How:** a dual-radio node connects to two APs on different bands (ideally 5 GHz plus 6 GHz) and sends every packet both ways. Receivers drop duplicates by sequence number.
- **Refinement, combined with idea 1:** the second path can carry the smaller "tight" layout (24-bit plus a 16-bit copy), saving bytes. Packet rate, the real Wi-Fi cost, is one per path either way.
- **Downlink:** the matrix sends to both of the node's addresses, through both APs.
- **Build notes:** different bands avoid the two radios interfering with each other. Antennas should be separated. Pi 5: onboard radio plus an M.2 or USB Wi-Fi 6E card.

3. Cable with a wireless safety net

- **How:** a node with both Ethernet and Wi-Fi sends on both, and the receiver keeps the first copy. Normally the cable wins.
- **Cheap insurance:** while the cable is up, only L0 and L1 go over the air. That's a fraction of the airtime, but the wireless link stays warm and ready.
- **Why:** on live events cables get kicked, crushed and unplugged. This turns a cable fault into a non-event.

4. Soft time slots on standard Wi-Fi

- **How:** all nodes share one clock, so the controller gives each node on an AP its own send offset within the 2 ms packet cycle. Nodes take turns instead of transmitting at the same instant.
- **Beacon-aware:** each AP sends a beacon about every 102.4 ms, which briefly blocks the air. Nodes read the beacon timing and schedule around it.
- **Downlink:** the matrix paces its sends to each AP the same way.
- **Limits:** devices outside our system still compete for air, so collisions are reduced, not eliminated. Phase 2 measures the gain.

5. Never send late audio

- **How:** each packet carries its playout deadline. The sender drops packets that can no longer arrive in time, so late packets never use airtime.
- **Linux nodes (Pi):** short driver queues (airtime queue limits), small socket buffers, and deadline-based send scheduling (`SO_TXTIME` with the ETF queue discipline). On a dedicated node, also a lower Wi-Fi retry limit.
- **Mac/Windows:** application-level pacing and small socket buffers.
- **Why:** on Wi-Fi, latency spikes come mostly from queues filling up behind a slow packet. Keeping queues near-empty cuts the worst-case delays.

Timing and sync

Every output plays at a known timestamp, so the system knows its own timing exactly. These ideas turn that into features.

7. Sync groups with click-free latency changes

- **How:** outputs that must stay phase-coherent (a PA array, delay towers, a broadcast stereo pair) form a sync group that shares one latency: the slowest member's requirement plus a margin. Outputs outside a group (in-ears, a talkback speaker) run as low as their own link allows.
- **Click-free changes:** when a link needs more latency, the group shifts by resampling under 0.1%, which is inaudible even on sustained tones. Shifting 1 ms takes about 1 s. Alternatively, it switches during detected silence.
- **Defaults:** PA system, delay towers, broadcast feed. In-ears are independent.
- **Why:** coherence where physics needs it, the lowest latency everywhere else, and latency that adapts mid-show without anyone hearing it.

16. Timecode and lip-sync everywhere

- **How:** the network clock locks to a house reference (GPS, PTP grandmaster, or word clock through a gateway). Any node can output LTC or MIDI timecode derived from it.
- **Lip-sync:** each broadcast output gets a video-delay compensation setting, so audio lines up with the camera path.
- **Why:** broadcast and shows driven by playback need audio, video, lighting and playback on one timeline. Usually that means extra boxes and cables; here it comes from the network.

17. Automatic speaker delay alignment

- **How:** a measurement-mic node captures a test signal (a sweep) from each speaker zone. Transport latency is known to the sample, so the remaining delay is pure acoustics. The system calculates and proposes delay times for each zone and its sync group.
- **Why:** measurement tools such as Smaart exist, but they have to guess the system's own latency. Here it's known exactly, so alignment is faster and more precise.

- **Show safety:** proposals only; nothing changes without operator approval, and never while the show is locked.

Reliability during the show

The live output must never stop, everything must be recorded perfectly, and every failure must be explainable afterwards.

6. Live and archive from one stream

- **How:** every node keeps a rolling buffer of everything it sends (all layers, at least 60 s; about 17 MB for 24-bit stereo, which is trivial). Live playout never waits. The recorder, which has no deadline, asks nodes to resend any packet or layer it missed.
- **Result:** the recording is bit-perfect 24-bit even when the live feed briefly ran at 16-bit or concealed a loss. Files are Broadcast WAV with network-time stamps, so all tracks line up.
- **Bonus, retroactive record:** because nodes buffer, "record" can capture the 60 s *before* the button was pressed.
- **Why:** live-recording, broadcast archive and post-production want perfect multitracks, and no wireless system delivers them.

13. Black-box incident recorder

- **How:** every node keeps rolling telemetry: arrival times, layers used, retries, signal strength, channel use, CPU, temperature. Any glitch or layer drop automatically saves a snapshot from both sides of the link.
- **Output:** an incident timeline with probable causes, for example: "21:43:12, Stage-L used L1 for 6 ms; retries spiked on channel 149 while AP-2 rescanned". Exportable as a post-show report.
- **Why:** for high-profile events, "what happened and why" must be answerable. It also turns every show into test data that improves the product.

14. Island mode

- **How:** each node has a configured fallback for total network loss:
  - Hold silence
  - Local passthrough (its inputs to its outputs, at a preset gain)
  - Play a preloaded safety loop or announcement
  - Fade out
- **In-ear nodes:** default to local mic passthrough, so a performer is never left deaf.
- **Reconnect:** re-sync, then crossfade back to network audio.
- **Why:** even the worst failure has a designed, predictable outcome.

15. Authenticated, encrypted audio

- **How:** every packet is authenticated and encrypted (AES-GCM, hardware-accelerated on Pi 5 and modern PCs, costing microseconds). Keys are issued by the controller when a node is adopted (idea 9). Unknown devices trying to join are alerted.
- **Interop boundary:** AES67, Dante and ST 2110 links stay unencrypted, as those standards require. Our gateways mark where encryption stops.
- **Why:** at a high-profile event, someone injecting audio into a PA or listening to private talkback is a real threat. Wireless makes it easier to attempt, so security must be built in, not added later.

Operations and setup

These ideas make a wireless system something an operator can plan, prove and trust before doors open.

11. Latency budgets and margin meters

- **Latency budget:** every route shows its full latency, broken down into converters, buffers, packet time, air time, jitter buffer and plugins. Operators see exactly where the milliseconds go.
- **Margin meter:** shows how close each link is to failing. Margin = the playout deadline minus how late the slowest packets arrive (the worst 0.1%). It reads like a signal meter: "4 ms margin" is safe, "0.5 ms" is not.
- **Routing by requirement:** the operator states what an output needs, for example "≤ 6 ms, coherent with the PA group, critical". The system picks the tier, paths and protection, or explains why it can't.
- **Why:** today's wireless gear shows "connected" right up until it fails. This shows how close to failure you are.

12. Show-ready stress test

- **How:** before doors, a test node floods our own network with simulated crowd traffic for 10–30 minutes. Every route's margin is checked, and the result is a green/red certificate.
- **Doors-open watch:** compares margins before and after the audience arrives. Bodies absorb 5 GHz signal and phones add noise.
- **Why:** problems are found at soundcheck, not during the headline act.

9. Stateless nodes, instant spares

- **How:** a node holds no config of its own. Its role (name, routes, gain, tier, keys) lives on the controller. A new or spare node shows a QR code or LED pattern, and the operator taps "adopt as Stage-L".
- **Spares:** keep pre-provisioned spares in the case. A dead node is replaced in under a minute.
- **Safety:** if a replacement's hardware differs (a different I/O HAT), the system warns before taking it live.

18. Walk-test site survey

- **How:** walk the venue with an Android phone or a battery-powered Pi node, tapping your position on a floor plan. The app records latency, loss and signal strength at each point and draws a heatmap.
- **Output:** recommended AP positions and channels, plus a saved venue profile for next time.

19. Capacity planner

- **How:** a model built from Phase 2 measurements, refined by every show's telemetry (idea 13). Enter nodes, channels, APs and tiers, and it predicts airtime use and margins.

- **Why:** answers "will 24 nodes on 3 APs work?" before quoting a job or loading the truck.

20. Show versioning and safe updates

- **Show versioning:** every change is already in the event log, so add named snapshots, diffs ("what changed since soundcheck?") and instant rollback.
- **Safe updates:** nodes keep two firmware copies and roll back automatically if an update fails. Updates are blocked while the show is locked.
- **Why:** the most common cause of show failures is a change nobody remembers making.

Reach and ecosystem

These ideas spread the protocol beyond our own hardware, which is where its long-term value lies.

21. Any Android phone as a node

- **How:** the Android app plus a class-compliant USB-C audio interface turns a phone into a wireless stereo I/O node. It has the same protocol, layers and sync, at whatever tier the device can manage (each device's latency is measured).
- **Uses:**
  - Field reporters and roaming presenters for broadcast
  - A quick extra node when a Pi isn't to hand
  - A wireless listening point anywhere in the venue
- **Why:** zero-hardware deployment. Every crew member already carries a potential node.

22. Open protocol and SDK

- **How:** publish the protocol spec and a reference SDK (Rust core, C API) so hardware makers can build compatible nodes: speakers, amps, stage boxes, wireless packs. A certification programme guarantees interop.
- **Business model:** this is the Dante playbook. Audinate's value lies mostly in licensing the protocol to thousands of products, not in selling its own boxes. Revenue comes from licensing, certification and our premium software (matrix, mixer, comms).
- **Why:** a protocol becomes a standard when other people build on it.

Lessons from other protocols

Most of what we need has been proven somewhere already: in wired AoIP, wireless mics, cellular networks or internet streaming. We borrow concepts, never proprietary wire formats.

Protocol / product	Concept worth borrowing	How we use it
AVB / Milan (IEEE TSN)	Stream reservation: a stream is only admitted if bandwidth is guaranteed. Presentation time. Certified device profiles.	Milan-style certification for the SDK (idea 22).
AES70 (OCA)	Open standard for device control and monitoring, used by Milan	**A gap in the current design:** adopt AES70 alongside NMOS and Ember+ for device control
Livewire+ (Telos Axia)	Each source can be sent as a low-latency stream and a standard stream; automatic "backfeed" mix-minus; GPIO over IP	Dual-latency streams per source (idea 23); confirms our automatic mix-minus
Dante Domain Manager	Domains, user roles, audit, device lock	Show domains: several shows or stages share one infrastructure, fully isolated (idea 28)
SMPTE ST 2110-31	Transparent AES3 transport, carrying non-PCM such as Dolby E bit-exact	Bit-exact transport mode for broadcast bitstreams
Shure Axient Digital	Frequency diversity (one mic on two frequencies at once); interference detection with automatic switch to a ranked backup frequency	Validates idea 2.
WiSA (wireless speakers)	About 5 ms latency and microsecond speaker sync on 5 GHz, using dedicated chips	Existence proof that 5 GHz plus scheduling reaches ~5 ms
DECT / DECT NR+	Continuous dynamic channel selection: each link keeps moving to its least-interfered slot	Soft time slots (idea 4) pick offsets by measured collisions, not a fixed table
Bluetooth LE Audio / Auracast	Broadcast isochronous audio to unlimited receivers, optionally encrypted	Assistive-listening output to hearing aids and audience phones (idea 24)
5G URLLC / private 5G	Pre-scheduled uplink with no request step; packet duplication across two carriers	Validates ideas 2 and 4. Private 5G as a future transport (already used for broadcast cameras).
802.11aa / WMM-AC	Multicast with retries for AV (GCR); admission control at the AP (TSPEC)	Use where APs support them; the wizard detects support
WebRTC (NetEQ, RED, FlexFEC)	Adaptive jitter buffer that time-stretches audio to conceal gaps and resize smoothly; redundant audio; flexible FEC	Better concealment than plain fades (idea 27); applied to the browser comms path too
SRT / RIST	Retransmission with a deadline (too-late packets dropped); bonding several internet links	Validates idea 5. Remote sites over the internet (idea 26).
JackTrip / SonoBus	Burg-algorithm packet-loss concealment, tuned for music	Concealment engine (idea 27)
OSC, MIDI 2.0, Art-Net / sACN	Show-control languages used by consoles, playback and lighting	Control API: OSC in and out; GPIO and logic triggers from lighting cues

New ideas from this review

23. Dual-latency streams per source

- **How:** a source can publish an ultra-low-latency stream (for in-ears and monitoring) and a robust higher-latency stream (for broadcast and recording) at the same time.
- **Why:** each listener picks its own trade-off from the same source, without extra hardware. Livewire proves broadcasters value this.

24. Assistive-listening output

- **How:** any matrix bus can go out as a Bluetooth Auracast broadcast to hearing aids and audience phones.
- **Why:** venues are increasingly required to provide assistive listening, so this turns a compliance cost into a feature.

26. Remote sites over the internet

- **How:** SRT/RIST gateways extend the matrix to remote venues and studios, with bonded links and their own latency tier. Mix-minus is automatic.
- **Why:** remote production (REMI) is how live broadcast increasingly works.

27. Music-grade concealment

- **How:** when every layer is lost, concealment uses Burg-algorithm prediction and NetEQ-style time-stretching instead of a fade to silence.
- **Why:** a 5 ms gap concealed this way is usually inaudible on music; a fade is not.

28. Show domains

- **How:** one physical network carries several isolated domains (Stage A, Stage B, broadcast truck), each with its own users, routes and keys. Shared sources cross domains only by explicit grant.
- **Why:** festivals and multi-room venues run several shows on one infrastructure without risk of crossed routes.

Network optimizer and software scheduler (design, 2026-10-10)

Rob asked: can the main matrix PC schedule nodes into slots like a custom access point would, with an AI-driven optimizer and a smarter setup wizard, maybe on the GPU? Answer: mostly yes, in software, no GPU needed; with clear limits.

What software on the matrix PC can and cannot control

	Can control	Cannot control
Our nodes' **send times**	Yes: every node shares network time, so the engine assigns each node a send offset within the period ("soft time slots", design idea 4; `SenderConfig::phase` already exists)	The radio's own backoff and retries once a packet is queued
Downlink release times	Yes: the engine chooses when each output packet goes to the AP (staggered, or burst-aligned so a Wi-Fi 6 AP can bundle them)	How the AP then queues and transmits them
Our nodes' **settings**	Yes, through a small agent on each node: power save off, background scans off (where the OS allows), voice priority (DSCP 46), second radio, buffer sizes	Settings the OS locks (some phones, iOS)
The **router's settings**	Through its admin API where one exists (OpenWrt/ubus, UniFi, Omada, enterprise controllers): channel, width, band steering off, airtime fairness, multicast rate, DFS avoidance	Routers with no API: the wizard tells the user exactly what to change
Other people's devices	Only by network design: dedicated SSID/channel, the venue's own router policy	Never by force: jamming or disconnecting other devices is illegal and off the table

What a software scheduler buys (and what it does not)

- It stops **our own** nodes colliding with each other. With N nodes all sending at the same instant each period, CSMA backoff serialises them and collisions cause retries; spreading them across the period removes most of that. The benefit grows with the number of nodes per AP.
- It cannot stop **outside** interference (neighbours, phones, microwaves): the projection (`11-latency-projections.md`) shows that is what sets latency at a strict glitch target. Hence the second radio path matters more than slots.
- Slot precision is limited by the nodes' OS timing: about 0.1-0.5 ms on desktop OSes, better on a Pi with a real-time kernel, best on an MCU node. Slots are therefore "soft" (a strong statistical gain, not a guarantee).
- **Capacity is airtime.** Each packet costs roughly 0.15-0.3 ms of airtime with overhead, so one AP carries on the order of 5-10 stereo nodes at 2 ms periods (uplink + downlink). Scheduling uses that airtime better; more APs on wired backhaul add capacity.

Compute: no GPU needed

Assigning slots for even a few hundred nodes, re-planned every few seconds, is microseconds of CPU work. The learning parts (per-link delay models, burst periodicity, choosing deadlines and tiers) are small statistical models that run comfortably on a CPU core. A GPU would only make sense for much heavier future ML (RF fingerprinting from many receivers), which nothing here needs.

The optimizer loop (runs locally on the matrix PC; never in the audio path)

1. **Measure, continuously:** per node RTT, one-way delay, jitter tail, loss runs, retries, signal and data rate (OS Wi-Fi APIs), late and concealed counts from the engine, channel survey (busy time, neighbour networks) from the PC and nodes.
2. **Diagnose:** match patterns to causes. Examples it can recognise: spikes every ~2 minutes (background scans), ~1 s periodic hops (macOS AWDL), 102.4 ms structure (neighbour beacons), DTIM-period delays (power save), a busy channel (survey), weak signal (rate drops), too many nodes per AP (airtime).
3. **Act, safely:** on our nodes directly (slots, settings, second radio, latency tier, per-input mix deadline); on the router via its API, with the operator's approval; otherwise plain instructions in the wizard. Never during a show without approval; every change logged and reversible.
4. **Verify:** keep a change only if the measured tail improves (A/B over a few minutes), else roll back.
5. **Explain:** an assistant (local model, or Morpheus when online) turns the diagnosis into plain words: "Laptop 3 scans for networks every 2 minutes, causing 40 ms spikes: turn off Location Services' Wi-Fi scanning, or use its Ethernet port."

Measurements just happen

Rob: "any field tests need to just happen or be button presses." So:

- **Button presses now:** `field/` holds double-click launchers (host on the wired machine, quick or full test on the Wi-Fi machine, a Termux:Widget button on Android). The test finds the host, records the Wi-Fi link itself and returns its report to the host (`amt-io field`).
- **Just happen, started:** the desk's Network panel (System and FOH) shows each input's delay, loss and first-copy share per path from live traffic, every 5 s. Next: every node and engine already measures live traffic (per-path delay and loss, headroom, outages, concealment, sync). The analyser collects these all the time, so a show's network report needs no test at all; "Test this network" in the wizard runs the field test from a button on the desk (engine as host, nodes as testers).
- Nothing that measures may need a terminal, an address typed in, or a file carried back by hand.

Build order

1. Soft slots: engine assigns `phase` per sender; measure the gain with a contention model in the simulator, then on real APs (E1/E9).
2. Downlink release timing (staggered vs burst-aligned), measured on APs.
3. Wizard v1: scans and checks (channel survey, neighbour networks, client power save, scans/AWDL detection from delay periodicity, DSCP), with instructions.
4. Node agent: applies node-side fixes with consent.
5. Router integrations: OpenWrt first (open), then the common prosumer APIs.
6. Optimizer loop with A/B verification; plain-language assistant.

RF location map: where the devices causing trouble are (design, 2026-10-10)

Rob: as much as possible from the network and RF, locate the devices causing issues and show them on Google Maps: a dot with the device's name, expanding to everything known about it.

What can be located, and how well

What	How	Accuracy (honest)
Our own nodes	Placed by the operator on the map (drag), GPS outdoors, refined by Wi-Fi ranging (802.11mc FTM round-trip time) between nodes and APs where supported	Exact if placed; FTM about 1-2 m
Access points (ours and neighbours')	Every node scans and reports each AP's signal strength; static, strong transmitters fit well	About 2-5 m indoors with 4+ nodes
Other Wi-Fi devices (phones, laptops, other systems on our channel)	Pi/Linux nodes with a second radio in monitor mode capture frame headers (never content) and report each transmitter's signal strength at each node; trilateration with a path-loss model calibrated from our own node-to-node signal strengths	Room level, about 3-8 m indoors; better with more nodes and FTM; phones randomise MAC addresses, so identity is per session
Non-Wi-Fi interference (microwave ovens, DECT, analog wireless kit, LED drivers)	No decodable frames: seen as channel busy time and noise-floor rise per node; located as "nearest node" and a heat map	Zone level; a spectrum-capable receiver (an SDR covering 2.4/5 GHz) can do better later

Each estimate is drawn with its uncertainty circle, never as a falsely exact dot.

What the dot shows when expanded

Name (our node's name; otherwise maker from the MAC prefix and a type guess: phone, laptop, AP, camera), MAC (noting when randomised), band and channel, airtime it uses (%), retry rate, signal at each of our nodes, estimated position and uncertainty, first and last seen, behaviour flags (periodic scanning, heavy airtime, an AP on our channel, power-save bursts), the effect on our audio (late/concealed periods correlated in time), and a suggested action (move our channel, ask the venue to turn it off, add an AP near the affected nodes).

Maps

- **Google Maps** (JavaScript API, satellite and road views) for outdoor events and large venues; needs internet and an API key with billing on Valiant's account.
- **Floor plans:** the venue's plan uploaded and pinned onto the map (a ground overlay), so indoor positions mean something.
- **Offline:** the floor plan alone, without Google tiles (AirMatrix works offline; the map must too).

Privacy and law (built in, not optional)

- Passive capture of frame **headers** only; payloads are never decoded.
- MAC addresses can be personal information (Australian Privacy Act, GDPR): kept on the matrix PC only, shown as maker plus a short hash, retained only for the show, with a venue notice. Never sent to the cloud.
- We never transmit to, disconnect or jam anyone else's device. Locating a problem is so the operator can act lawfully (change channel, ask the venue).

Build order

1. Our nodes on the map, coloured by link health (from telemetry we have).
2. APs from node scans, trilaterated.
3. Monitor-mode capture on Pi nodes (second radio): other Wi-Fi devices.
4. FTM ranging where hardware supports it.
5. Non-Wi-Fi interference heat map.
6. Google Maps view and floor-plan overlay in the control UI (web work that Morpheus can take on).

Nodes, capacity per access point, range and wireless backhaul (2026-10-10)

Software nodes (PC, Mac, Android) and their sources

Every software node can offer, as sources:

- **Sound-card inputs** (built-in or USB audio interface).
- **"Internal": whatever the device itself plays back**:
  - Windows: WASAPI loopback capture of any output (no special permission).
  - macOS: Core Audio process taps (macOS 14.2+) or ScreenCaptureKit audio (13+); the user grants permission once.
  - Android: AudioPlaybackCapture (Android 10+); the user approves a capture prompt each session; apps can opt out (some streaming apps do) and phone calls cannot be captured.
  - Linux / Pi: the PipeWire or PulseAudio "monitor" of the output.
  - iOS: not possible system-wide (iOS stays web-only, as decided).
- **The device's microphones** (comms "hot mic" and talkback):
  - Pick which mic (phones have 2-4; Android lists them with direction), or a headset/USB mic.
  - Use the unprocessed or "voice performance" input where offered, so the OS's own gain control and noise suppression do not fight ours.
  - Our own **mic manager**: input meter, digital gain, "auto set level" (say a few words, it sets the gain to a target level with headroom), high-pass, limiter, optional noise suppression, mute, push-to-talk or latched hot mic. Most phones do not expose analogue mic gain, so gain is digital (and the source is usually 16-bit).
- Android also takes **USB audio interfaces** (USB Audio Class over OTG), which makes an old phone plus a cheap interface a usable node. Latency varies a lot by phone; the node reports what it measures. Android's low-latency Wi-Fi mode (no power save, no scans while in use) is switched on by the node app.

Microcontroller boards

- **ESP32-DevKitC-VIE** (Rob's DigiKey find): original ESP32 = 2.4 GHz Wi-Fi 4 only. **Not suitable** for audio nodes (5 GHz needed).
- **ESP32-C5-DevKitC-1**: dual-band (2.4/5 GHz) Wi-Fi 6, about US$14 (DigiKey); Seeed XIAO ESP32-C5 about AUD 10 ex GST (Core Electronics, Australia). The pocket-node target.
- ESP32-C6: Wi-Fi 6 but 2.4 GHz only (not suitable). ESP32-S3: 2.4 GHz Wi-Fi 4 (not suitable for the radio; fine as a control/UI chip).
- NXP RW612: dual-band Wi-Fi 6 MCU with I2S; pricier module, an alternative if the ESP32-C5's audio clocking falls short.

Capacity per access point (`amt-chain --capacity`)

Per channel at 2 ms packets, safe 40% airtime:

	one-way stereo streams	two-way nodes
Standard router (single user), typical link, 80 MHz	4.8	2.4
Custom AP with OFDMA, 80 MHz, 8 x 106-tone RUs, MCS7	**12.8**	**6.4**
Same, strong link (MCS9)	14.7	7.4
Custom AP with OFDMA, 20 MHz (2 RUs)	3.3	1.6

AirMatrix suits OFDMA unusually well: every node finishes its period at the same moment, so the AP can collect all of them in one scheduled transmission. OFDMA needs **wide channels** (80 MHz); on 20 MHz it is worse than plain Wi-Fi. One AP carries one audio channel per radio band: a Wi-Fi 6 dual-radio AP gives one 5 GHz channel outdoors; a tri-band (Wi-Fi 6E/7) AP indoors gives 5 GHz plus 6 GHz, about **12 two-way nodes or 25 one-way streams per AP** with OFDMA. Whether a given AP chipset lets us schedule this way is the custom access point lab's question.

Frequencies, rules and range (Australia)

- Wi-Fi 6 runs on 2.4 GHz and 5 GHz; Wi-Fi 6E adds 6 GHz; Wi-Fi 7 uses all three and can use two at once (multi-link).
- **6 GHz is licence-free but indoor-only in Australia** (lower band 5925-6425 MHz, about 250 mW; the 2025 LIPD class licence added 6425-6585 MHz). Outdoor events use 5 GHz (and 2.4 GHz as a last resort).
- 5 GHz limits vary by sub-band (200 mW indoor-only in 5250-5350 MHz, up to 4 W EIRP for 5725-5850 MHz); radar-sharing (DFS) sub-bands can force a channel change, so the planner prefers non-DFS channels for audio.
- **Range for audio is shorter than "Wi-Fi range"**: we need a strong link, because a weak link drops the data rate and the same packet then takes 2-4x the airtime (capacity falls accordingly). Planning rule to validate in E1: nodes within about 10-15 m of an AP indoors (line of sight where possible), APs above head height (bodies absorb 5/6 GHz), more APs rather than stretching range. 6 GHz reaches less far than 5 GHz.

Wireless backhaul instead of cable (Ubiquiti and similar)

- Use **60 GHz point-to-point bridges** (802.11ad/ay) as "cable" between APs, or from the matrix PC to a remote AP: Ubiquiti UniFi Building-to-Building Bridge (about 1.7 Gbps, up to 500 m, 5 GHz backup radio) or airFiber 60 LR (long range, 12+ km). 60 GHz does not use the 5 GHz audio spectrum and its narrow beams barely interfere.
- **Avoid 5 GHz point-to-point or mesh backhaul for audio APs**: it competes with the audio channels and adds jitter.
- No published latency figures: **measure** with `amt-probe` and `amt-sync` (E14: hours of probing over the bridge, with people walking through the beam and in rain).
- When a 60 GHz link is blocked it falls back to its 5 GHz radio (lower rate, shared spectrum, and the switch itself may glitch). Treat a bridge as **one of two paths**: the other a cable or a second bridge on a different route, with AMT's two-path delivery making a switch inaudible.
- AMT-Sync runs through a bridge like through a cable; the measured delay and its asymmetry go into the sync calibration.

15. Roaming between access points

Rob (2026-10-10): "How are we handling when someone moves between access points? Is that going to affect the radio paths? Do we need to look at it?" Yes: it is the one everyday event that stops a radio carrying packets for far longer than any of our protection covers.

The problem

A Wi-Fi client that moves to another access point disassociates and reassociates (and re-keys). On consumer routers the gap is typically **50-300 ms**; with fast roaming (802.11r) tens of ms. Scanning for a better access point also takes the radio off its channel for tens of ms per channel scanned. Our in-packet redundancy covers two periods (4 ms), the jitter buffer a few more; a roam on a single radio is an audible dropout.

Packets the client sent during the gap wait in its own queue and go out in a rush afterwards: they arrive *late*, not lost.

What we do

1. Two radios roam one at a time (make before break)

Every packet already goes over both radios (multipath, E15). A roam on one radio is harmless while the other carries the packets. The danger is both radios leaving at once, which is exactly what operating systems do when a presenter walks away from both access points: each radio's signal fades at the same moment and each roams on its own schedule.

`amt_node::roam::Coordinator` decides instead, once a second:

- a radio may **scan or roam only while every other radio has been healthy** (associated, signal at least -75 dBm, its path delivering) for 3 s, and since its own last scan or roam;
- **roam** when the signal is under -68 dBm and an access point of the same network **on the same band** is at least 8 dB stronger (each radio keeps its band, so the two stay independent), never more often than every 10 s;
- otherwise **scan** a weak radio at most every 20 s (a scan is a short gap on that radio);
- weakest radio first; one action at a time; an action that does not finish in 5 s is given up on;
- **locked** nodes (stage boxes, front of house, anything that does not move) never roam;
- a **single radio** has nobody to cover it: it acts only when nearly unhealthy anyway, and the receiver treats the gap as an outage.

"Delivering" comes from the sync answers over each radio (AMT-Sync runs over every path), so the coordinator knows a path works end to end, not just that the link is up.

Simulated (`node_end_to_end`, two radios, 150 ms roam each, 12 ms latency): **together: 48 periods concealed; one at a time: nothing concealed, nothing late, no outage seen at the receiver.**

Platform: `amt-io send --roam-control [--roam-dry-run]` on Linux (Pi, Linux laptops), with every `--to` pinned to an interface and one `--follow`. Reads `iw dev IF link` and `wpa_cli scan_results`; acts with `wpa_cli scan` and `wpa_cli roam BSSID`. **Not yet run on a Pi.** To settle on hardware:

1. Stop the supplicant roaming and background-scanning on its own (per network `bgscan=""`; under NetworkManager the equivalent settings), so only the coordinator moves radios.
2. Whether `wpa_cli` reaches the supplicant under NetworkManager (Pi OS Bookworm), or the D-Bus interface is needed instead.
3. Roam time with and without 802.11r on Rob's routers.

Windows and macOS give applications little roaming control (Windows: the adapter's "roaming aggressiveness"; macOS: none). The wizard sets what it can and the two-radio test (E16) tells us how often both fade together anyway. Android: no control; phones are single-radio nodes.

2. Receivers know an outage is not lateness

A roam on a single-radio node (or both radios at once) stops arrivals, then the queued packets arrive late. Automatic latency used to learn from those late packets: raise 1 ms and remember, so every roam ratcheted the latency up a notch for nothing (no latency short of the outage would have saved them).

Now the receiver's network thread counts an **outage** whenever no audio packet at all arrives for more than 20 ms (`OUTAGE_GAP_NS`); lateness from a too-low latency never stops arrivals, so the two are told apart. For 3 s after an outage automatic latency ignores lateness and headroom. Simulated: a 150 ms roam at 150 s: one outage, 74 late packets, latency unchanged (11.57 ms); without the rule it went to 12.57 ms.

3. The network must be one network

All access points must bridge into **one subnet** (one Layer 2 network), so a roaming client keeps its IP address and its flows. Mesh systems do this; separate routers usually do not (each hands out its own addresses). The wizard checks it (a node's address and gateway before and after a roam).

4. Fast roaming on the access points

Where the access points support **802.11r (fast transition)**, plus 802.11k (neighbour reports, so scans are short) and 802.11v, turn them on: they cut a roam from hundreds of ms to tens. Some older clients refuse 802.11r networks; the wizard warns. This shortens single-radio gaps; it does not replace two radios.

5. Sync through a roam

A roam changes the path's delay and asymmetry. AMT-Sync's level-shift detection handles the single-path case (`05-clock-sources.md`); with two radios the other path keeps answering, and losing a path causes no step (tested).

Still to do

- E16 walk test on Rob's rigs (`docs/experiments/E16-roam-walk-test.md`).
- The platform items above, on a Pi with a second USB radio.
- Wizard checks: one subnet, 802.11r/k/v, roaming thresholds, lock stationary nodes.
- The engine side: a node roaming changes which access point the engine's downlink packets go through; nothing to do on a bridged network, but measure it in E16.

16. Built-in channel processing

Rob (2026-10-10): "We will need to build in all the standard compression, dynamics, EQ, reverb and delay per channel in the mix engine, so a great mix is possible out of the box; that way a Pi can be the full backup."

Third-party plugins (VST3/AU/CLAP, Waves) stay available on the main engine, but most have no Linux ARM builds, so a Raspberry Pi backup engine cannot run them. With our own processing for everything a typical show needs, the Pi carries the same mix, and nobody has to buy plugins to get a good one.

What is built (`crates/amt-dsp`)

Per input flow, a stereo-linked **channel strip** between the jitter buffer and the matrix, in console order:

Stage	What	Notes
Trim, polarity	-24..+24 dB	polarity flips glide through zero (no click)
Gate	threshold, range, hysteresis, attack, hold, release	release fades in dB (even fade to the floor)
EQ	high-pass 12 or 24 dB/oct + 4 bands (peak, shelves, low/high-pass)	RBJ biquads, f64 inside; matches its drawn curve to 0.05 dB
Compressor	threshold, ratio, soft knee, attack, release, makeup, peak/RMS	log-domain, stereo-linked, gain-reduction meter

On **aux sends** (a pair of matrix outputs in, a pair of matrix inputs back): **reverb** (8-line feedback delay network, Householder mixing, diffusion, pre-delay, decay time, treble damping, size, width) and **delay** (up to 2 s, feedback, damping, ping-pong; time changes glide like tape). The return reaches the mix one period (2 ms) later, inaudible on a wet signal.

Every setting changes click-free (filters glide their coefficients; the stable region of a biquad is convex, so the glide is always stable), nothing allocates while processing (tested with an allocation trap), and silence cannot slow it down (no subnormal numbers).

Output-bus processing (`amt_dsp::output`, built 2026-10-12): each output flow runs through its own EQ (the same biquad set) and a stereo-linked peak limiter, in that order, between the matrix mix and the packet. The limiter attacks instantly (a sample never passes the ceiling) and releases with a one-pole, so it is a true ceiling with no click; it reports its current gain reduction for the meter (design 18). Defaults are flat EQ and limiter off, so an output changes nothing until an operator turns it on. Set from the desk or the control API like a strip (`{"op":"output","output":j,"params":{...}}`) and mirrored to a backup engine with every other change.

Presets (`amt_dsp::strip::preset`): `speech`, `vocal`, `music`, `flat`: identical on every engine, main or backup. `amt-io engine --strip IN:preset` and `--reverb-send IN:dB` (one shared reverb, returned to every output without a mix-minus).

Measured

- EQ: peak and shelf gains to 0.01-0.3 dB of the setting; high-pass -3.01 dB at the corner, 12/24 dB per octave; the processed audio matches the curve to 0.05 dB.
- Compressor: static curve exact (hard and soft knee), attack and release time constants within 2 %.
- Limiter: a steady level above the ceiling settles at the ceiling and its gain-reduction meter reads the exact reduction; a full-scale step never passes the ceiling; turning it off releases with no step. Through the engine, a full-scale tone into a -6 dBFS ceiling sits at -6 dBFS on every period's peak and the meter reports the reduction.
- Reverb: decay time within 15 % of the setting (0.8, 1.8, 4 s), left and right tails decorrelated ($|r| < 0.3$), treble decays faster with damping. Lengths are whole samples once settled: reading between samples is a gentle low-pass that, round the network, had made a "bright" 2 s tail lose its treble in 1.1 s (found and fixed).
- Live, through the engine, vocal preset plus reverb send on a 997 Hz tone: 103 dB (the compressor's detector ripple on a steady tone; EQ and reverb are linear). Without processing: 132 dB.
- **CPU** (the test Mac (Intel, macOS)'s 2015 i7-5775R, one core): a full console of 64 mono strips with every stage on 21.7 % (0.34 % each), 4 stereo reverbs 3.7 %, 4 stereo delays 0.7 %: **26 % of one core**. A Raspberry Pi 5 core is in the same class and it has four: a Pi can carry the whole show.

Main and backup

The backup engine (Rob's stage Pi) runs the same strips and effects with the same settings, mirrored from the main engine. Outputs will not be bit-identical (a packet late at one engine and not the other, and libm differences between x86 and ARM in coefficient maths), so receivers switch engines with a short crossfade rather than mixing packets from both.

Next

- The mixer UI: strip editing, EQ curve, meters (`StripMeters`, `eq_magnitude`).
- Mono strips per matrix input (today: per stereo flow, linked).
- Sidechain and ducking (presenter over music), de-esser, tap tempo for the delay.
- Settings mirroring to the backup engine; the backup engine itself.
- A listening session with Rob on real voices and music.

17. Comms panels: built-in camera tally

Rob (2026-10-10): "Camera comms panels should have built-in tally that picks up the signal from the switcher. Most switchers these days have that GPIO available digitally, so those are the ones we need to accept first; then an external GPIO box can go under hardware builds to integrate later."

What it does

A camera operator's comms panel (browser panel or hardware panel, see the comms section of the spec) shows the camera's **tally**: red when its switcher input is on air (programme), green when it is in preview. The engine knows which switcher input each camera is (from the panel's role template: "Camera 3 = switcher input 3"), listens to the switcher, and pushes tally state to every panel over the existing control connection. Optional: drive a tally light from the panel device (phone torch, LED on a hardware panel), and show tally on the producer's and director's panels for every camera.

Phase 1: digital tally from the switcher (software only)

In order (most coverage first):

1. **TSL UMD v3.1 and v5.0** (UDP/TCP): the broadcast standard. Ross, Grass Valley, Sony, Panasonic, Vizrt/TriCaster, many others (and most tally routers) speak it.
2. **Blackmagic ATEM** (its network protocol): the most common switcher at events and small broadcast.
3. **vMix** (TCP API, `TALLY` events) and **OBS** (obs-websocket): software switchers at smaller shows and streams.
4. **NDI tally** and **Roland V-series** (LAN), **Ember+** (Lawo) as needed.

Each is a small adapter that turns the switcher's messages into one internal form: `(switcher input, programme, preview)`; the panels never care which switcher it was. The setup wizard finds switchers on the network where it can (ATEM, vMix and OBS announce themselves or listen on known ports) and asks which input each camera is.

Design points:

- Tally must be **fast and dependable**: push on change, refresh every second (a missed message heals), show "tally lost" on the panel if the switcher goes quiet, never a stale red.
- No cloud: the switcher, the engine and the panels are on the same network.
- Read-only: we never send commands to the switcher.

Phase 2: external GPIO box (hardware builds, H4)

For switchers that only offer contact-closure tally (GPIO pins), a small box with opto-isolated inputs (8-16 channels) turns closures into the same internal tally messages over the network: ESP32-class board, Ethernet or Wi-Fi, powered over USB or PoE. Listed in `10-hardware-builds.md` as H4.

Status

Design only. Comms panels themselves are later in the roadmap (the protocol and matrix come first); tally is part of the first comms build.

18. Metering and measurement

Rob (2026-10-10): "Add audio engineering test tools: RTA with scrolling spectrogram, VU and PPM meters, loudness meters, vector scope / goniometer, all the things they will want, pro-level great-looking readouts we can even do things with to help tune the system. General health and the engineering readouts in the network analyser and setup wizard; the operational audio stuff integrated into the mix engine."

Where each tool lives

Mix engine (operational: the operator watches while mixing)	Network analyser and setup wizard (engineering: set up, tune, check health)
Peak with hold, true peak	Path delay per radio, loss, first-copy share (`amt_node::paths`)
PPM (DIN, Nordic, BBC, EBU)	Headroom readout, automatic latency status
VU (reference level set per show, e.g. -18 dBFS = 0 VU)	Sync state, per-path round trips, offsets
Loudness (EBU R128 / BS.1770-4): M, S, I, LRA, true peak, max M	Roam events, outages, Wi-Fi link stats
Phase correlation and balance	Concealment and late packets per input
Goniometer / vector scope	**Transfer function** (magnitude, phase, coherence)
RTA (1/3 octave) and scrolling spectrogram on any channel or bus	**Delay finder** (align delay towers and fills)
Gain reduction (gate, compressor) per strip	RTA and spectrogram for room and noise checks

One library (`crates/amt-meter`) serves both, so a reading means the same thing everywhere.

What is built

- **Loudness** (`loudness`): K-weighting from the filters' analogue design (matches the BS.1770 coefficients at 48 kHz to 1e-9); momentary, short-term, integrated with absolute (-70 LUFS) and relative (-10 LU) gates; loudness range (EBU Tech 3342); true peak (4x oversampled). Histograms keep memory fixed for a show of any length. **Passes the EBU Tech 3341 steady-tone and gating cases to 0.1 LU and the Tech 3342 range cases to 1 LU.**
- **Level** (`level`): sample peak with hold; PPM types with self-calibration (a 5 kHz burst reads its specified amount under the steady tone to 0.05 dB; a steady 1 kHz tone reads its level); VU with 300 ms to 99 % and 1.5 % overshoot, 0 VU set by a reference level. *The PPM figures are the commonly published ones for IEC 60268-10 types: check them against the standard's text before claiming compliance.* Sample-based peak and PPM readings sit up to ~0.5 dB low on high tones (samples miss crests); true peak does not.
- **Stereo** (`stereo`): correlation (-1..+1), balance, goniometer point stream (side across, mid up; mono is vertical).
- **Spectrum** (`spectrum`): FFT analyser calibrated in dBFS; 1/3-octave RTA (IEC centres 20 Hz-20 kHz), with a decimated path for bands under 160 Hz so low bands have real resolution; pink noise reads flat within 1.5 dB from 25 Hz to 16 kHz; spectrogram rows on a log frequency axis.
- **System tuning** (`transfer`): dual-channel transfer function with coherence (shows a +6 dB EQ peak to 0.3 dB, coherence > 0.95); delay finder (GCC-PHAT, sub-sample: 437 samples found to 0.2, a half-sample delay to 0.25).
- **Gain reduction** (read back in the desk's meter snapshot, built 2026-10-12): the gate, compressor and output limiter each report their current gain reduction (dB) as they process — the strip's compressor on the strip, the limiter on the output panel. A meter reads what is happening now, not a setting, so a limiter that is not working reads zero.
- **Cost**: the full operational set (peak, PPM, VU, loudness with true peak, correlation, goniometer) is 0.65 % of one 2015 i7 core per stereo channel, true peak being most of it: full set on outputs and buses, peak and PPM on inputs. Spectrum, RTA and transfer function run off the audio thread.

Doing things with the readings (system tuning)

The point of measurement is action. Planned wizard flows built on these:

1. **Delay alignment**: play programme or pink noise, measure each fill or delay tower with a mic; the delay finder gives the delay to set; apply it on that output (and check with the transfer function's phase).
2. **EQ assist**: the transfer function, where coherence is high, shows the system's response; propose output EQ moves (never on low-coherence regions, which are room reflections or noise, not the speaker).
3. **Gain structure**: per input, peak and loudness history suggest trim so channels sit at a target (e.g. -18 dBFS average, peaks under -6).
4. **Loudness compliance**: programme outputs against a target (-23 LUFS EBU R128, -24 LKFS ATSC A/85, -16 / -14 LUFS for streams), with true-peak limits, logged for the show report.
5. **Network health** beside audio health: path, sync and concealment readouts in the analyser, so "is it the network or the mic?" has an answer.

Next

- The display: the desk shows input peak meters with hold, the output's loudness and the gain-reduction meters, and the **Meters page** (built 2026-10-12) shows every mix's R128 loudness, a loudness history graph, and the input peak/GR bank — all a view over the event-stream snapshot, no new DSP. Still to do: the Meters page's PPM/VU choice, goniometer, RTA and spectrogram, and the Tuning page (transfer function, delay finder). Design with Rob.
- A lock-free tap from the audio thread to the analysis thread per selected channel or bus.
- Surround loudness weights; 1/6 and 1/12 octave smoothing; multi-time-window transfer function; per-output EQ to apply the assist's suggestions.

19. Anyone's stage boxes: audio in and out, and control

Rob (2026-10-10): "We should be able to use anyone's stage boxes too. It would be great to incorporate control for those boxes: with raw AES67 we get the audio from Dante but can't control the input gain or other features. It's important that we can input and output live, low-latency, full broadcast audio to and from any stage box that works on a network."

Audio: AES67 (and ST 2110-30) on the wired side

The engine is wired; stage boxes are wired. So the engine speaks the industry's interoperable format to them natively, and AMT (our Wi-Fi protocol) only for wireless nodes. The engine bridges the two, with every box's inputs as matrix sources and outputs as matrix destinations.

What that needs (PTPv2, AES67 receive and send, SAP built 2026-10-10, see E17; NMOS not yet). AES67 outputs carry the on-time mix, so for box outputs feeding PA or in-ears, set the mix delay to cover the inputs' arrival.

- **PTPv2** (IEEE 1588, AES67 media profile) as follower, and as grandmaster when nothing else leads; tie AMT network time to it (AMT-Sync already has an external-reference class, `05-clock-sources.md`).
- **AES67 RTP streams**, receive and send: L24 at 48 kHz, 1 ms packets (also 125-333 µs for low latency where boxes allow), up to 8 channels per stream, multicast with IGMP; ST 2110-30 is the same audio with stricter timing rules.
- **Discovery**: SAP/SDP (what Dante's AES67 mode uses) and AMWA NMOS IS-04 (registry) / IS-05 (connection management) for broadcast plants.
- **Latency**: box to engine is one wired hop: 1 ms packets plus a jitter buffer of 1-2 ms. Fine for live and broadcast; in-ears through the engine stay within the budget in `09-latency-plan.md`.

Dante boxes in AES67 mode, what to expect (third-party summaries, mostly older; check Audinate's current notes and each box's firmware): multicast only, in 239.69.0.0/16; 48 kHz and 24-bit only; up to 8 channels per flow; primary network only (no Dante redundancy); SAP discovery; PTPv2; sends 1 ms packets, receives shorter; non-standard DSCP (clock CS7, media EF). Not every Dante chipset supports AES67 mode (older Ultimo-based boxes may not).

Native Dante (no AES67 mode needed, unicast, Dante Controller sees us as a device) needs an Audinate licence (Dante Embedded Platform, or their software products). A business decision for Rob; worth a conversation with Audinate once the product is real.

Control: preamp gain, phantom, pad, high-pass

There is no common standard in practice. Dante carries audio, not preamp control; consoles control their own vendor's boxes over proprietary protocols (e.g. Yamaha consoles and Rio, A&H consoles and DX/GX, some third-party preamps by device ID from Yamaha consoles). The open standards exist but are not yet common on stage boxes:

- **AES70 (OCA)**: open device control, including preamp gain; adoption on stage boxes is thin (forum requests for it on Dante boxes are unanswered).
- **AMWA NMOS IS-12**: device control and monitoring over WebSockets and JSON, with standard and vendor-specific models; aimed at broadcast plants.
- **Ember+**: widely used in broadcast (Lawo and others); open-ish.

Plan: a driver layer, one driver per protocol, each mapping a box's controls to the same internal model (per channel: gain, phantom, pad, high-pass, input select, meters; per box: clock, status, temperature):

1. Open standards first: AES70, NMOS IS-12, Ember+.
2. Documented vendor APIs next, box by box (several makers publish TCP or UDP command sets for their I/O; confirm each before promising it).
3. Proprietary console protocols only with the maker's permission or SDK (partnership, not reverse engineering: legal risk and fragile).
4. Where no control exists: the box is still fully usable for audio; our channel strip's trim gives digital gain, and the wizard warns that analog headroom is set on the box itself.

Shared boxes (front of house and monitors on one box): when either side changes analog gain, the other side's digital trim compensates so its mix does not jump (gain compensation, as pro consoles do). Needs the driver to report gain changes, which the good protocols do.

Our own stage box

H2 (Pi 5 stage box, `10-hardware-builds.md`) speaks AMT natively, so its gain and phantom are ours to control, with no gateway.

Order of work

1. PTPv2 follower + AES67 receive (box inputs into the matrix): unlocks anyone's boxes for audio in.
2. AES67 send (matrix outputs to box outputs), SAP announce, IGMP.
3. NMOS IS-04/05 for broadcast plants.
4. Control drivers: AES70, IS-12, Ember+, then vendors by demand.
5. Test kit: a Dante box in AES67 mode, a Ravenna/AES67 box, Rob's gear.

Sources (research 2026-10-10)

- Allen & Heath forum, Dante and AES70 control: https://forums.allen-heath.com/t/dante-and-aes70-control/32572
- Gearspace, digital stageboxes for live recording: https://gearspace.com/threads/digital-stageboxes-for-live-recording.1083245/
- AMWA IS-12: https://specs.amwa.tv/is-12/ and https://specs.amwa.tv/is-12/latest/docs/Overview.html
- NMOS tutorial, NAB 2022 IP Showcase: https://potato.vsf.tv/events_archive/2022-04_NABShow2022/NMOS%20Tutorial%20-%20NAB22%20IP%20Showcase.pdf
- Dante AES67 limits: https://www.svconline.com/?p=112371 ; DirectOut AES67 real life (NAB 2019): https://potato.vsf.tv/events_archive/2019-04_NABShow2019/Tuesday/1000%20Claudio%20Becker-Foss%20-%20IPShowcase_DirectOut_AES67_RealLife.pdf ; Merging, Dante devices in AES67 mode: https://merging.atlassian.net/wiki/spaces/PUBLICDOC/pages/2001341/Configure+MERGING+and+third+party+DANTE+devices+in+AES67+mode

20. Backup engine (a Pi on stage, or any second machine)

Rob (2026-10-10): a secondary mix engine, e.g. a Pi 5 on stage, that is right there to keep everything running if the main engine drops out.

How it works

- **Both engines mix the same show.** Every input node sends to both (the two-path sending already does this: `send --to MAIN:PORT --to BACKUP:PORT`, or each engine subscribes to the same AES67 stage box streams). Both run the same routes and built-in processing (design 16), so a Pi can carry the whole show without third-party plugins.
- **Same network time.** Both follow the same AMT-Sync or PTP leader, so their outputs are sample-aligned: period `n` of the main engine's mix and period `n` of the backup's are the same audio.
- **Receivers take over, seamlessly.** A receiver plays the main engine's flow and also buffers the backup's (`ReceiverConfig::backup_ssrc`; `receive --ssrc MAIN --backup-ssrc BACKUP`; the backup engine runs with `engine --ssrc-base 0x0B000000`). Before rendering it looks a little ahead (2 ms past what it is about to render): if the main flow is missing audio there that the backup has, it crossfades to the backup over 2 ms, which finishes before the gap. The main flow's concealment is never heard. It returns once the main flow has been complete for 2 s, with the same crossfade.
- **Not packet by packet.** The two engines' outputs are not bit-identical (one may have had an input packet late that the other did not; maths libraries differ between x86 and ARM), so receivers switch with a fade rather than mixing packets from both.

Measured

- Simulation (`node_end_to_end`, main engine dead 8-10 s): 998 periods concealed in the main flow, none heard; switched over once and back once; every window after start-up > 95 dB; timing within 5 us.
- Live on the Mac: tone sender to two engines, receiver with both flows; main engine stopped at ~10 s: switched to the backup, 0 concealed in what played, tone 132.4-132.8 dB straight through.

Control API and mirroring (built 2026-10-10)

`engine --control PORT` serves the engine's control API: JSON, one object per line over TCP (routes, strips by preset or full settings, effects, get the whole state, set the whole state; see `amt-io/src/control.rs`). It is also what the mixer UI will speak. `engine --mirror-to BACKUP:PORT` keeps a connection to the backup's control port: on every (re)connect it sends the whole state, then each change as it is applied. Tested: a backup that comes up late catches up at once, then gets each change. Live: a -12 dB route change sent only to the main engine reached the backup; when the main stopped, the receiver switched and the level held (-24.0 LUFS before and after; an unmirrored backup would have jumped back up 12 dB).

Still to do

- Leader backup: AMT-Sync election and hitless takeover exist (design 05); configure the backup engine as the backup time leader.
- AES67 outputs: a stage box takes one stream; for box outputs, the backup engine's stream needs the box's own redundancy (ST 2022-7) or a switch on our side. Wi-Fi nodes (AMT) already switch.
- The Pi build itself (hardware H2) and a failover drill in E-series.

21. Blackmagic Design integration

Rob (2026-10-10): "We will need to build integrations for Blackmagic broadcast systems too: this system fits their cost effectiveness and may be a really good option to use together for live broadcast and events."

Blackmagic's market (ATEM switchers, HyperDeck recorders, Videohub routers, 2110 IP converters, cinema cameras) is exactly the cost-conscious live and broadcast production AirMatrix targets. The integration has two halves: audio in and out over the network, and control.

Audio: ST 2110-30 through the 2110 IP converters

Blackmagic's 2110 IP Converters (3x3G, 8x12G and others; RJ-45 or SFP) carry embedded SDI audio (up to 16 channels) to and from ST 2110 IP, with ST 2110-30 at **conformance Level C** (48 kHz, 1-8 channels per stream, 1 ms packets) and **PTP to IEEE 1588-2008 with the SMPTE ST 2059-1/-2 profile**. That is what our AES67 receive and send already do (design 19, E17), with two settings to respect:

- **PTP domain**: the ST 2059-2 profile uses domain 127 by default (AES67's media profile uses 0). `--ptp follow --ptp-domain 127` (and the same for `lead`). Our follower takes any Sync interval and the two-step, end-to-end mechanism; ST 2059-2's timing is within that.
- **Streams**: ST 2110-30 requires a zero media clock offset (ours sends 0) and Level C means at most 8 channels per stream at 1 ms: our `--aes67-out` streams to a converter should use `/2`, `/4` or `/8`. Several streams for more channels.
- **Discovery**: ST 2110 plants use NMOS IS-04/05 more than SAP. Until we have NMOS (design 19), load the converter's SDP as a file (`--aes67 stream.sdp) and give it ours (Stream::to_sdp`).

So: an ATEM's programme audio (or any SDI embedded audio) can come into the AirMatrix engine through a converter, and AirMatrix mixes can go back into SDI (to the ATEM, recorders, the truck) the same way. To verify on real hardware (a converter on loan): lock, channel mapping, Level C limits, and the offset rule.

Control and workflow

Product	Interface	What we would do with it
ATEM switchers	ATEM Switchers SDK (official; the network protocol itself is not published; community reverse-engineered libraries exist but break with firmware)	**Tally** to comms panels (design 17); **audio follows video** (raise a presenter's mic when their camera goes to programme, duck others); read the ATEM's Fairlight mixer levels; programme/preview state on the desk
HyperDeck recorders	HyperDeck Ethernet Protocol (published, text over TCP; also RS-422)	Record and stop with the show, slate clip names from the scene, read timecode, record AirMatrix mixes and the ISO feeds together
Videohub routers	Videohub Ethernet Protocol (published, text over TCP)	Keep labels in step with the matrix, show video routes on the desk, audio follows a Videohub route
Cameras	Camera control over SDI (documented in Blackmagic's SDKs); talkback over SDI via ATEM and the talkback converters	Bridge the ATEM's camera talkback into AirMatrix comms, so camera operators on SDI and crew on Wi-Fi share channels
2110 IP converters	ST 2110 and its control (NMOS where supported)	Audio as above; later, route control through NMOS

Order of work

1. **Audio through a 2110 IP converter** (works with today's code; needs the domain setting and a hardware test).
2. **ATEM tally and audio-follows-video** through the official SDK (check its licence terms allow shipping it inside a commercial product before building on it; the published text protocols need no SDK).
3. **Videohub label and route sync**, **HyperDeck record control** (both published protocols: build and test against the manuals, then a unit).
4. **Camera talkback bridge** with the comms build.

Notes

- Port numbers commonly quoted for HyperDeck (TCP 9993) and Videohub (TCP 9990) are to be confirmed from the current manuals, not assumed.
- Blackmagic's SDKs are under their own licence (not open source): allowed under our rules only if redistribution in a commercial product is permitted; check before linking anything. The published protocols are safe to implement directly.

Sources (research 2026-10-10)

- 2110 IP converter specifications (ST 2110-30 Level C, PTP 1588-2008 / ST 2059-1/-2, 16 channels embedded): https://www.huss-licht-ton.de/print_product_info.php/en/products_id/55538 , https://www.adorama.com/blackmagic-design-2110-ip-3x3g-converter/p/bmd3x3g , https://www.georges.com.au/products/blackmagic-2110-ip-converter-3x3g
- ST 2110-30 and AES67 differences, Level C limits (Blackmagic forum, 2024): https://forum.blackmagicdesign.com/viewtopic.php?p=1039683
- HyperDeck developer information and Ethernet protocol: https://www.blackmagicdesign.com/au/developer/products/hyperdeck/overview
- ATEM SDK: https://blackmagicdesign.com/uk/developer/products/atem/sdk-and-software ; forum on the ATEM network protocol: https://forum.blackmagicdesign.com/viewtopic.php?p=676464